Starke Kapslung der Interna Und Deprecations

Starke Kapselung

- Nahezu alle Abhängigkeiten (Frameworks, Libraries, JDK-APIs, eigene Projekte) bestehen aus:
    - **öffentlichen, stabilen APIs**
    - **internem Code**, der die öffentliche API unterstützt
- Ziel: **Vermeidung unbeabsichtigter Nutzung interner APIs**
- Ergebnis: **robustere, besser wartbare Projekte**
- Wir betrachten:
    - Warum das notwendig ist
    - Was genau interne APIs sind (v. a. im JDK)
    - Wie starke Kapselung in der Praxis funktioniert

Starke Kapselung

- OpenJDK-Codebasis wird laufend **refaktoriert**: Code wird verändert, verschoben, gelöscht
- **Öffentliche API bleibt stabil** – sie ist der Vertrag mit Java-Nutzern
- Trennung **öffentliche API ↔ interne Implementierung** ist entscheidend:
    - interne Nutzung kann bei jedem Minor-Update brechen
    - kann **JDK-Upgrades blockieren**
- Gleichzeitig bieten interne APIs manchmal einzigartige Fähigkeiten
- Lösung:
    - **Mechanismus, der interne APIs standardmäßig sperrt**,
    - aber **selektiv freigeben** lässt → **starke Kapselung**

Starke Kapselung

- Nur **exportierte oder geöffnete Packages** eines Moduls sind zugänglich
- Alles andere gilt als **intern und unzugänglich**
- Seit **Java 9** ist das JDK in **Module** aufgeteilt
- Folge: Interne APIs werden **standardmäßig verborgen**

Project Jigsaw – Überblick (Java 9)

Ziel: Langfristige Modularisierung der Java-Plattform und von Anwendungen

- **Kernbausteine:**
  - module-info.java zur Definition von *Exports* und *Dependencies*
  - *Module Path* als Ersatz/Ergänzung zum Classpath
- **Vorteile:**
  - Klare **Kapselung**: Nur explizit exportierte Packages sind sichtbar
  - **Stärkere Integrität & Sicherheit** des Codes
  - **Schnellere Startzeiten** und kleinere Runtime-Images durch maßgeschneiderte JREs (jlink)
  - Verbesserte Wartbarkeit großer Codebasen

Project Jigsaw – Überblick (Java 9)

Auf die Plattform selbst angewandt:

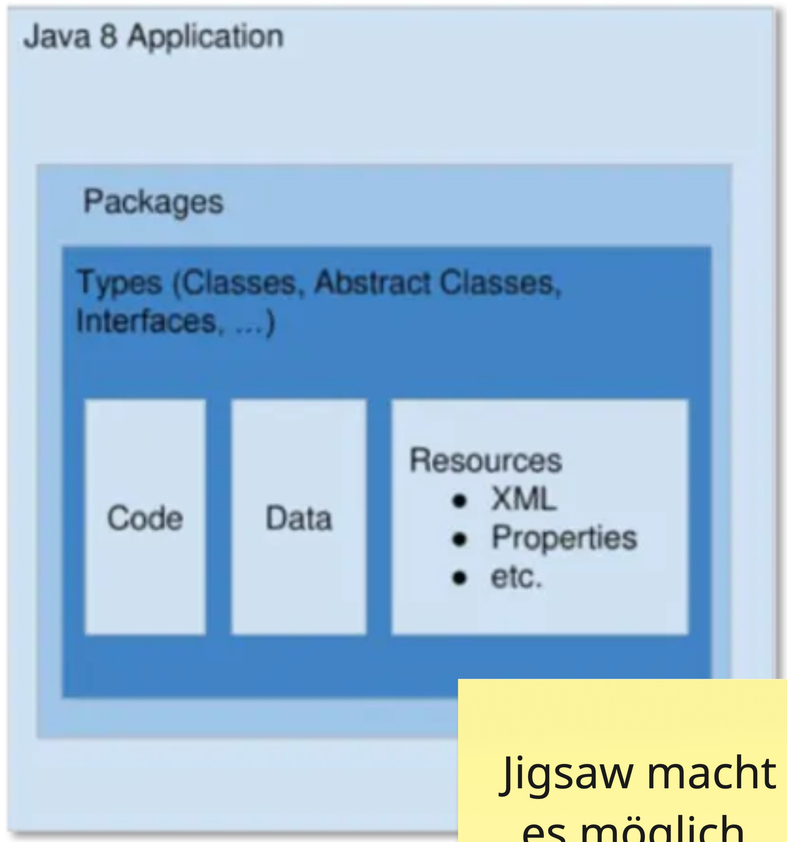
- Das JDK wurde in über 90 Module zerlegt (z. B. java.base, java.sql, java.desktop)

Praktische Relevanz:

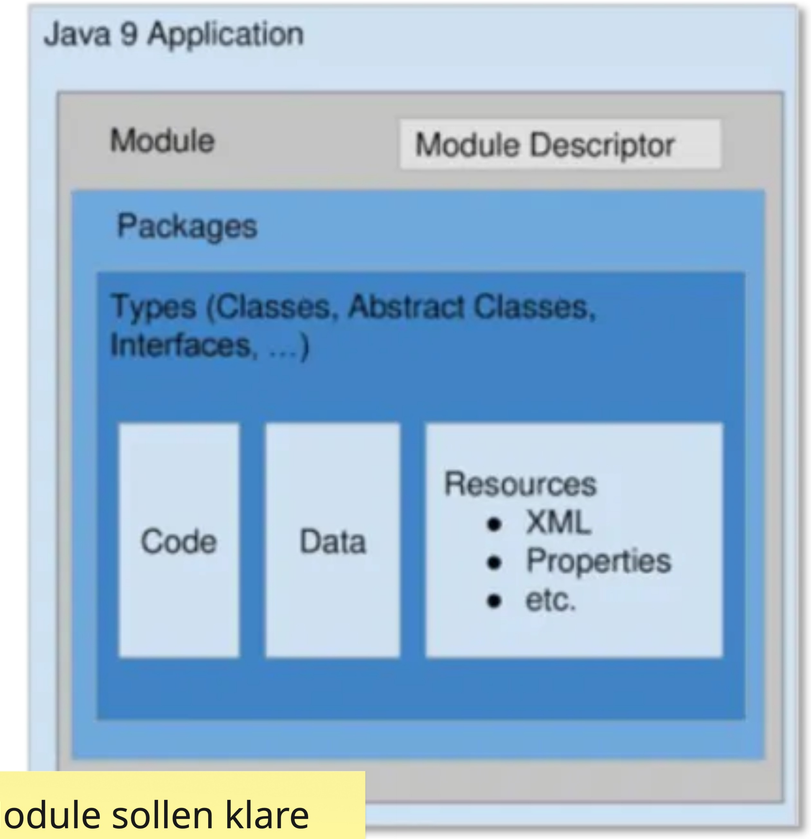
- Für Frameworks und große Anwendungen nutzbar, aber in vielen Projekten weiterhin begrenzt eingesetzt
- Wichtig für die interne Struktur des JDK und Tools wie jlink und jmod

De facto sind weiterhin Maven und Gradle der Standard

Vom Classpath zu Modulen



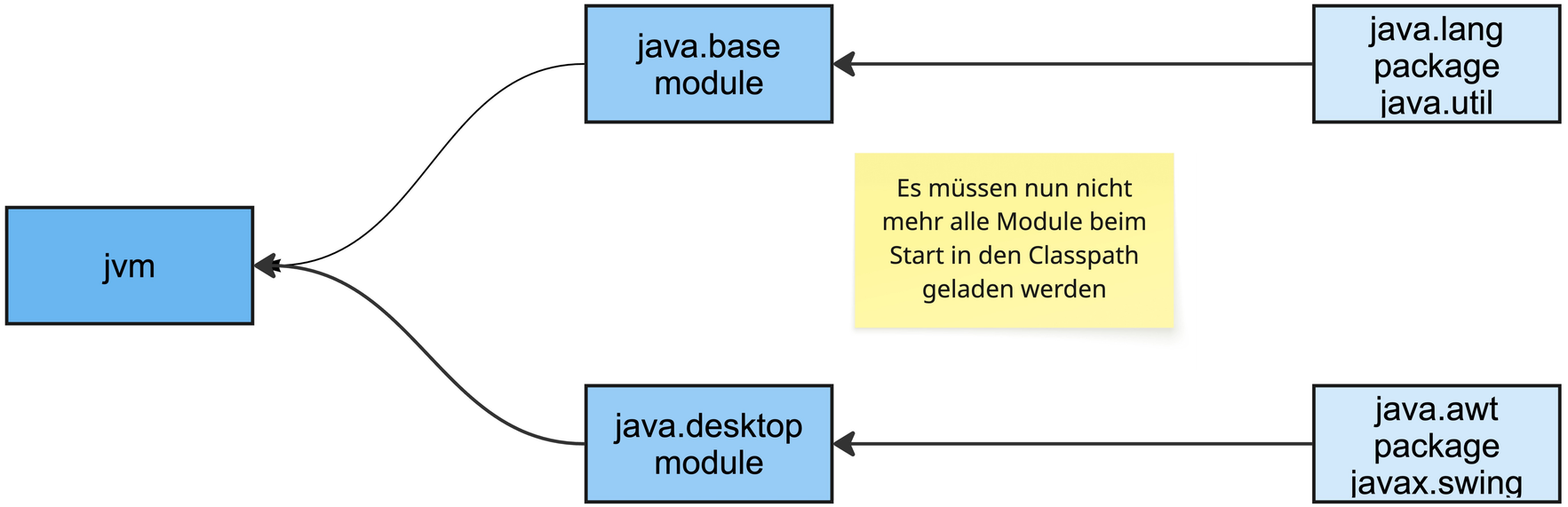
Jigsaw macht es möglich, den Classpath aufzuteilen



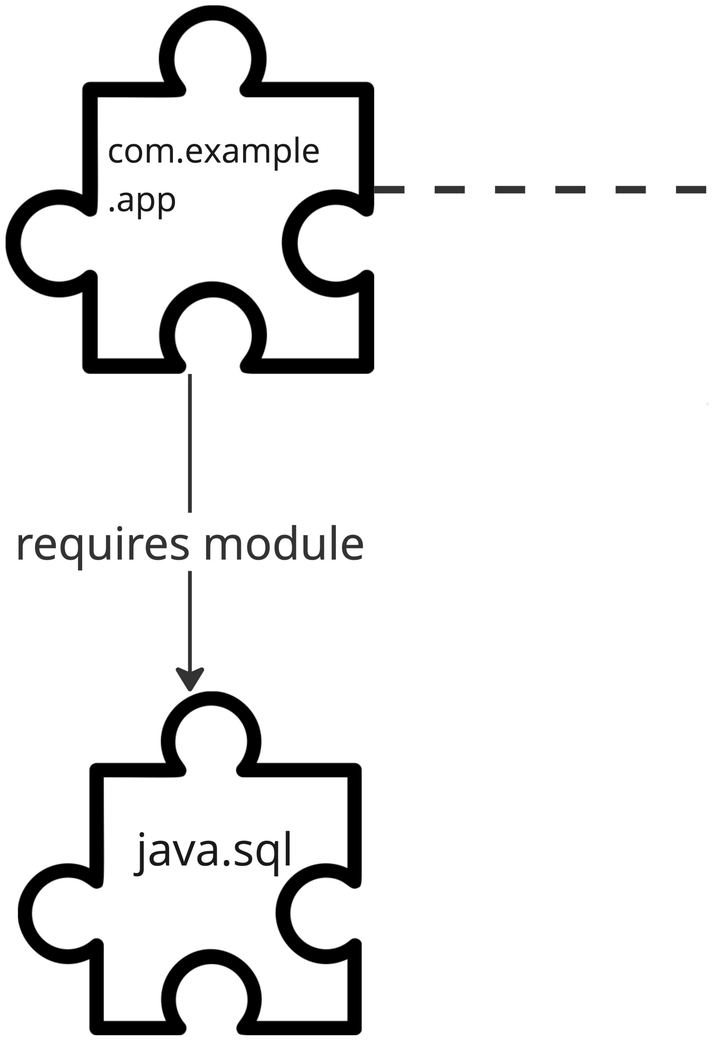
Module sollen klare Separierung zwischen den Teilen des JDK bieten

Bis Java 8

Seit Java 9



module-info.java




```
module com.example.app {
    // Wir benötigen Funktionen aus java.sql und einem eigenen Modul
    requires java.sql;
    requires com.example.util;

    // Wir geben nur das API-Package nach außen frei
    exports com.example.app.api;

    // Services können optional deklariert werden
    uses com.example.util.LoggingService;
}
```

Wichtige APIs

`java.*`

- öffentliche API
- aber: nur **öffentliche Member öffentlicher Klassen**
- weniger sichtbare Klassen/Member sind **intern**
- Modul-System kapselt sie stark

Wichtige APIs

`sun.*`

- **fast vollständig intern**
- Ausnahmen:
    - sun.misc.* und sun.reflect.*
    - vom Modul **jdk.unsupported** exportiert/geöffnet
    - bieten kritische Funktionalität (z. B. sun.misc.Unsafe)
- Grundregel: *sun. vermeiden**

Wichtige APIs

`com.sun.*`

- **JDK-spezifisch**, nicht Teil des Java-Standards
- ca. **90 % intern** (nicht exportiert)
- ca. **10 % exportiert** durch jdk.*-Module → dürfen verwendet werden
- Exportierte Pakete werden **kompatibel weiterentwickelt**

Beispiele

Das folgende Beispiel kompiliert ohne Probleme:

```
public class Internal {
    public static void main(String[] args) {
        IO.println(java.util.List.class.getSimpleName());
    }
}
```

Das folgende schlägt mit einem Compile-Error fehl.

```
IO.println(sun.util.PreHashedMap.class.getSimpleName());
```

Beispiele

Auch wenn man durch Reflection versucht, dies zu umgehen, erhält man dann zur Laufzeit einen Fehler.

```
Class.forName("sun.util.BuddhistCalendar").getConstructor().newInstance();
```

```
Exception in thread "main" java.lang.IllegalAccessException:
    class Internal cannot access class sun.util.BuddhistCalendar (in module java.base)
    because module java.base does not export sun.util to unnamed module @1f021e6c
        at java.base/jdk.internal.reflect.Reflection.newIllegalAccessException(Reflection.
        at java.base/java.lang.reflect.AccessibleObject.checkAccess(AccessibleObject.java:
        at java.base/java.lang.reflect.Constructor.newInstanceWithCaller(Constructor.java:
        at java.base/java.lang.reflect.Constructor.newInstance(Constructor.java:480)
        at org.codefx.lab.internal.Internal.main(Internal.java:9)
```

jdeprscan

Um potentielle Probleme durch Deprecations zu ermitteln, gibt es das Tool **jdeprscan** im JDK.

```
~/r/g/j/a/jdeprscan-practice ❯❯❯ jdeprscan --release 21 target/jdeprscan-practice-0.0.1-SNAPSHOT.jar
JAR-Datei target/jdeprscan-practice-0.0.1-SNAPSHOT.jar:
class tech/erben/java17/jdeprscan/LegacyNewsFeed erweitert die veraltete Klasse java/util/Observable
class tech/erben/java17/jdeprscan/LegacyNewsFeed verwendet die veraltete Klasse java/util/Observable
class tech/erben/java17/jdeprscan/LegacyNewsFeed hat eine Methode namens subscribe mit dem veralteten Parametertyp java/util/Observer
class tech/erben/java17/jdeprscan/LegacyThreadController verwendet die veraltete Methode java/lang/Thread::stop()V (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacyThreadController verwendet die veraltete Methode java/lang/Thread::suspend()V (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacyThreadController verwendet die veraltete Methode java/lang/Thread::resume()V (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacySecurity verwendet die veraltete Klasse java/lang/SecurityManager (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacySecurity verwendet die veraltete Methode java/lang/System::setSecurityManager(Ljava/lang/SecurityManager;)V (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacySecurity verwendet die veraltete Methode java/lang/System::getSecurityManager()Ljava/lang/SecurityManager; (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacySecurity hat eine Methode namens installBasicManager mit dem veralteten Rückgabetyp java/lang/SecurityManager (forRemoval=true)
class tech/erben/java17/jdeprscan/LegacySecurity hat eine Methode namens currentManager mit dem veralteten Rückgabetyp java/lang/SecurityManager (forRemoval=true)
```